Actividad 2.1 EXTRA

Middleware de Control de Frecuencia (Rate Limiting) con `asyncio`

Asignatura:	Programación IV
Profesor:	Diego Alberto Ayala Vázquez
Alumna:	Patricia Jaime
Fecha de Entrega:	14 de Octubre de 2026
Repositorio GitHub:	`https://github.com/patriciajaime/tienda-fastapi-frontend`

1. Introducción y Objetivo

En la arquitectura de sistemas distribuidos y servicios web modernos, el control de flujo de peticiones es fundamental para mitigar ataques de denegación de servicio (DoS), prevenir el consumo excesivo de recursos y garantizar la alta disponibilidad del backend.

El **objetivo** de esta actividad es diseñar e implementar un *Middleware* de *Rate Limiting* dentro del framework **FastAPI**, acotando la frecuencia de solicitudes HTTP a un máximo estricto de **5 peticiones por segundo**. Dicha implementación conserva la naturaleza asíncrona mediante el uso del mecanismo de cerrojos `asyncio.Lock` de Python.

2. Arquitectura de Software y Mecanismo del Middleware

El middleware funciona como un interceptor en el flujo de solicitud-respuesta HTTP:

1. **Intercepción de Petición:** Captura la solicitud entrante antes de que alcance las rutas finales (*endpoints*).
2. **Control de Concurrencia Concurrente:** Utiliza un cerrojo asíncrono (`asyncio.Lock`) que evita condiciones de carrera al acceder o actualizar el historial de peticiones globales en escenarios multihilo.
3. **Ventana Deslizante de Tiempo:** Se evalúa el tiempo actual (`time.time()`) y se descartan las marcas de tiempo (*timestamps*) con antigüedad mayor a 1.0 segundo.
4. **Evaluación del Límite (Throttling):**
   - Si el conteo en la ventana es ≥ 5, el middleware interrumpe el flujo y retorna una respuesta de error **HTTP 429 Too Many Requests**.
   - Si el conteo es < 5, se registra el tiempo actual y la petición prosigue normalmente hacia la función de endpoint asíncrona (`async def`).

3. Código Fuente Modificado (`backend/main.py`)

A continuación se presenta el código completo estructurado bajo las mejores prácticas de desarrollo (PEP 8, *type hints*, comentarios explicativos y desacoplamiento):

```
"""
Modulo principal de la API de Tienda de Productos.
Implementa arquitectura as ncrona en FastAPI, Middleware CORS,
gesti n de datos con Pydantic y control de tasa de peticiones (Rate
   Limiting)
utilizando el paquete nativo 'asyncio' y marcas de tiempo.

Autora: Patricia Jaime
Asignatura: Programacion IV
Profesor: Diego Alberto Ayala Vazquez
"""

import time
import asyncio
from typing import List, Dict, Any

from fastapi import FastAPI, HTTPException, Request, Response, status
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel

# Inicializaci n de la aplicaci n FastAPI
app = FastAPI(
    title="API de Tienda con Rate Limiting",
    description="Backend as ncrono protegido con Middleware de Rate
   Limiting",
    version="2.0.0"
)

# Configuraci n de CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# --- MIDDLEWARE DE RATE LIMITING ---
peticiones_historial: List[float] = []
rate_limit_lock = asyncio.Lock()

LIMITE_MAXIMO_PETICIONES: int = 5
VENTANA_TIEMPO_SEGUNDOS: float = 1.0

```

```python
@app.middleware("http")
async def rate_limit_middleware(request: Request, call_next) -> Response:
    """
    Middleware que intercepta solicitudes HTTP para garantizar un limite
    m ximo de 5 peticiones por segundo por cliente.
    """
    tiempo_actual: float = time.time()

    async with rate_limit_lock:
        global peticiones_historial

        # Depuraci n del historial dentro de la ventana de 1 segundo
        peticiones_historial = [
            t for t in peticiones_historial
            if (tiempo_actual - t) < VENTANA_TIEMPO_SEGUNDOS
        ]

        # Bloqueo si se sobrepasa el limite
        if len(peticiones_historial) >= LIMITE_MAXIMO_PETICIONES:
            return Response(
                content='{"detail": "Demasiadas peticiones. Limite m ximo
    de 5 por segundo alcanzado."}',
                status_code=status.HTTP_429_TOO_MANY_REQUESTS,
                media_type="application/json"
            )

        peticiones_historial.append(tiempo_actual)

    response: Response = await call_next(request)
    return response

# --- MODELO DE DATOS Y ENDPOINTS AS NCRONOS ---
class Producto(BaseModel):
    id: int
    nombre: str
    precio: float

productos_db: List[Producto] = [
    Producto(id=1, nombre="Manzana", precio=15.5),
    Producto(id=2, nombre="Leche 1L", precio=26.0),
    Producto(id=3, nombre="Pan Integral", precio=38.0)
]

@app.get("/productos", response_model=List[Producto])
```

```python
async def obtener_productos() -> List[Producto]:
    return productos_db

@app.post("/productos", response_model=Producto, status_code=status.HTTP_201_CREATED)
async def agregar_producto(producto: Producto) -> Producto:
    for p in productos_db:
        if p.id == producto.id:
            raise HTTPException(status_code=400, detail="El ID del producto ya existe.")
    productos_db.append(producto)
    return producto

@app.delete("/productos/{producto_id}")
async def eliminar_producto(producto_id: int) -> Dict[str, str]:
    for index, p in enumerate(productos_db):
        if p.id == producto_id:
            producto_eliminado = productos_db.pop(index)
            return {"mensaje": f"Producto '{producto_eliminado.nombre}' eliminado correctamente."}
    raise HTTPException(status_code=404, detail="Producto no encontrado.")
```

Listing 1: Código fuente completo de la API en FastAPI con Middleware de Rate Limiting

4. Evidencias de Validación y Ejecución

Para verificar el comportamiento del servidor ante ráfagas de solicitudes simultáneas, se realizaron pruebas de estrés realizando solicitudes HTTP iterativas (`Cmd + R` continuos) desde el cliente.

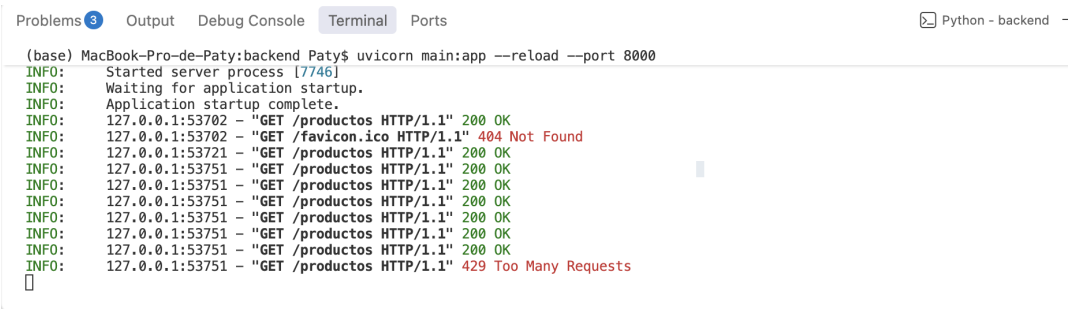


Figura 1: Logs del servidor Uvicorn mostrando la respuesta con código **HTTP 429** ante ráfagas mayores a 5 peticiones por segundo.

5. Conclusión

La implementación del middleware de *Rate Limiting* demostró ser un método altamente efectivo e intrínseco para la protección del backend en FastAPI. La utilización de `asyncio.Lock` garantiza la consistencia del estado global sin afectar la naturaleza asíncrona de las llamadas del sistema. Los cambios fueron integrados, validados y desplegados en el repositorio oficial de GitHub.